Chapter 2: Users, Requirements, and Constraints

Understanding a problem gives us a place to start. People building a solution together need a shared sense of what they are trying to accomplish and enough agreement to choose useful work.

That agreement can be surprisingly incomplete. Everyone may approve a proposal for “a better volunteer signup system” while imagining different things. One person expects automatic reminders. Another expects to approve every signup. A developer assumes that choosing a shift immediately reserves it.

All three can leave the same conversation thinking the decisions are settled.

Requirements help us make those expectations visible. A requirement describes a capability or condition the solution needs to satisfy. It might concern something a person can do, such as canceling a shift. It might concern who can see information or how reliably a confirmation is saved. The useful question is whether the statement helps the people involved understand what they can expect.

We will continue with the campus club from Practical Problem Solving. Its volunteers sign up through a group chat, while the organizer keeps a separate list. Changes have been missed, and at least one shift has gone uncovered. Suppose the club now wants to explore a small application that keeps commitments in one place. We still need to determine what that application should do.

Enough understanding to begin

Trying to produce a complete set of requirements before starting can stop a project altogether. People disagree about details, discussions repeat, and the prospect of building anything gets further away. Eventually everyone is frustrated and discouraged. The effort to eliminate uncertainty has become a reason to do nothing.

“The perfect is the enemy of the good” applies here. Often it is better to agree on some general requirements and get going. We can learn things from an early piece of working software that another round of discussion would not reveal.

For the club, agreement that everyone needs to see the same schedule may be enough to begin exploring that part of the application. The exact wording of reminder messages can wait. Whether signups need approval matters before implementing confirmation, but it need not prevent every other piece of work.

The question is what we need to understand for the work we are about to do. Keep open questions visible and revisit them as they become relevant. We will examine how to work this way more closely later in the course. For now, the requirements in this chapter are a way to get useful work moving, and they can develop as we learn.

Users are part of a larger group

A **user** interacts with the software. A **stakeholder** has an interest in the result or is affected by it. The two groups overlap, but they are not identical.

Volunteers and the organizer would use the signup application. The club treasurer might approve spending without ever opening it. Next year’s officers would inherit the responsibility for keeping it useful. An event coordinator might depend on the table being staffed while having no interest in the application itself.

Those people bring different questions. A volunteer wants to know whether a shift is available. The organizer needs to see gaps before the event. The treasurer needs to know whether operating the application will cost money. Next year's organizer needs access that survives this year's officers graduating.

The person asking for the software may represent several of these interests, but we should not assume that they understand every experience. Someone who organizes an event every month may forget how confusing the first signup was. A frequent volunteer may know shortcuts that a new member has never seen.

We also need to distinguish a role from a particular person. An organizer can volunteer for a shift too. That does not mean every volunteer should be able to edit the whole schedule. Roles describe what someone needs to do in a situation; they are not necessarily permanent categories of people.

A useful early check is to ask who supplies information, who depends on the result, and who handles trouble. Someone usually ends up answering the message when a confirmation is wrong. That person's work belongs in our understanding of the system.

Finding out what people need

Your first description of a problem is a starting point for investigation. People leave things out because they seem obvious, because they rarely happen, or because they do not know the answer themselves.

Finding those missing pieces is often called **requirements elicitation** (1). The term describes work we have already begun: talking to people and examining how things happen now.

For the club, the existing chat and signup list are useful evidence. We can follow a signup request from the first question and find out whether it ever became a confirmed commitment. Where does the organizer copy information? How does a cancellation arrive? What happens when somebody asks for a shift that has already been filled?

Questions about actual events tend to produce better detail than questions about an imagined perfect application. "Tell me about the last cancellation" can reveal a problem that "Would you like a cancel button?" never reaches.

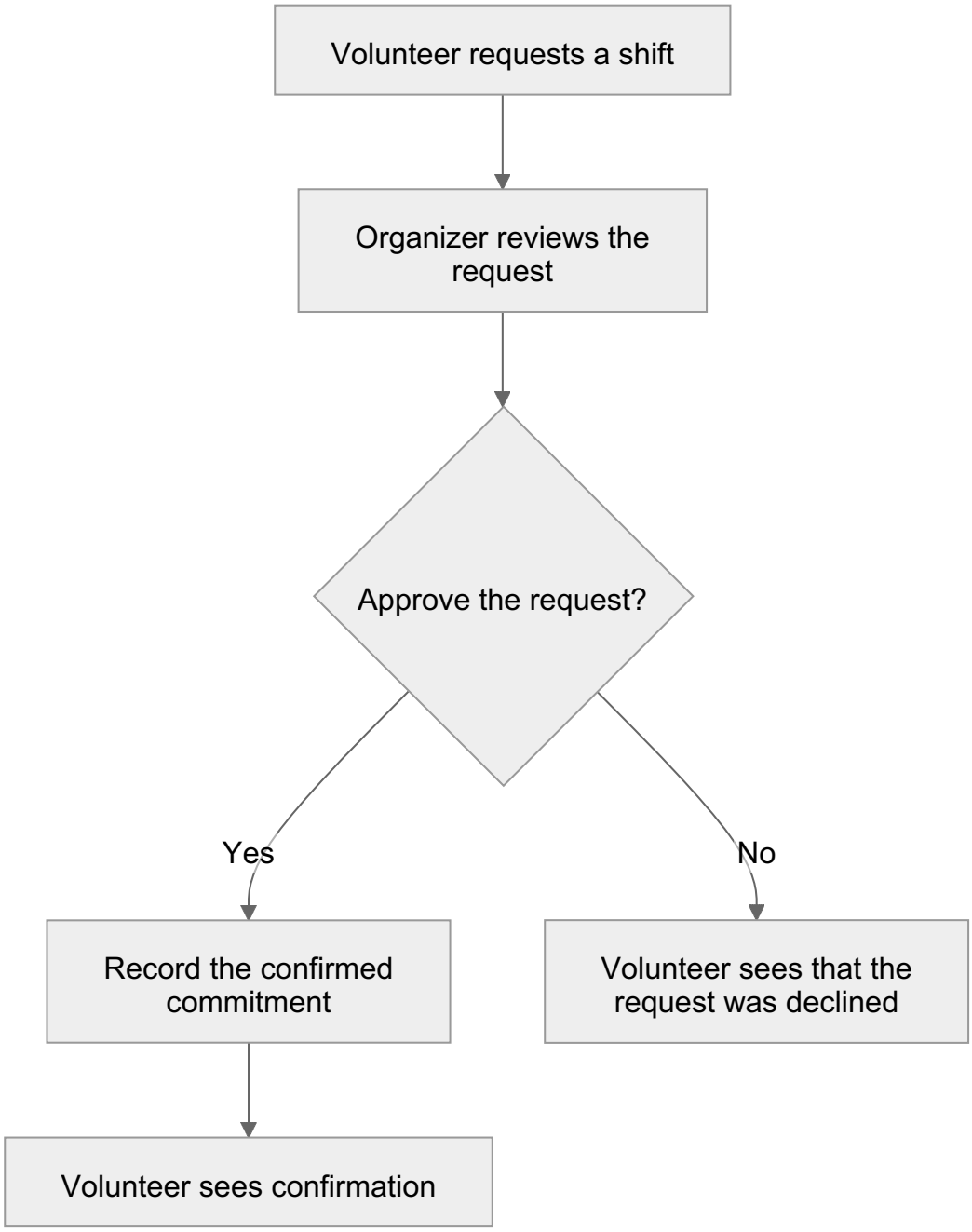
Observation adds something that an interview may miss. The organizer might say that updating the schedule takes only a moment. Watching the work could reveal that finding the right message and checking the other list takes much longer than the update itself. That extra work may have become so familiar that the organizer no longer thinks to mention it.

Sometimes a sketch helps. We could draw a shift with an available place, then show what it looks like after someone signs up. If the organizer asks, "Where do I approve them?", the sketch has exposed a decision. We have learned something before implementing either interpretation.

A sketch is limited evidence. Someone understanding a screen on paper does not establish that they can use the finished application on a phone. Different kinds of investigation answer different questions.

A workflow makes the decisions visible

A **workflow** shows how an activity proceeds (2). This proposed signup workflow assumes that the organizer reviews requests. That is one possible arrangement for the club, and drawing it gives everyone something concrete to question.



The arrows mean “what happens next.” The diamond marks a decision, and both outcomes need an understandable result. Submitting a request does not mean the volunteer has a confirmed place.

The chart also exposes questions. Who checks that a place is still available? What happens while the organizer has not yet answered? If the club decides that approval is unnecessary, this workflow changes. We can discuss that now without building the approval feature first.

User stories and story cards

A **user story** describes a small piece of useful behavior from the perspective of someone who needs it. It connects an aspect of a requirement to a person's experience. A common form is "As a [role], I want [capability], so that [benefit]." (3)

For the club, we could write:

> As a volunteer, I want to see which shifts still need help so I can choose one where I am useful.

The role tells us whose experience we are considering. The capability identifies something they need to do. The reason helps us judge whether our proposed solution would actually help. "Add a database table" might be implementation work, but it does not tell us what the volunteer gains.

The wording is a useful aid, not a form that makes a requirement correct when every blank is filled. "As a user, I want a better system so things are better" tells us very little. Naming the volunteer's decision gives us something to examine.

A card keeps the conversation focused

A **story card** is a short written record of the story, on paper or in a digital tool. It keeps the need visible while people work out the details. The card supports a conversation and an agreement about how to recognize a satisfactory result; it does not replace either one. (4)

Here is a possible card for the club:

Story card: Find a shift that needs help	Details
Who	A volunteer choosing where to help
Need	See which shifts still have unfilled places
Reason	Choose a shift where another volunteer is needed
Example	Saturday, 9-10 a.m., needs two more volunteers
Open question	Do pending requests count against the available places?

We have enough to explore the display of staffing gaps. We have not settled how requests reserve capacity. Keeping that question on the card makes the uncertainty visible without holding up every part of the application.

Walk through one person's experience

Suppose Alex opens the schedule and sees that Saturday's 9-10 a.m. shift needs two more volunteers. Alex chooses it and submits a request. Under the proposed approval workflow, the application shows that the request was saved and is waiting for review. After approval, Alex sees the confirmed commitment with its date and time.

These are different results. A saved request is not yet a confirmed place. Walking through Alex's experience exposes that distinction more clearly than "support signups."

Situation	What Alex should be able to observe
Choosing where to help	The Saturday shift has two unfilled places
Request submitted	The request is saved and awaiting review
Organizer approves	The commitment is confirmed for the specified date and time

The first row illustrates the story on the card. The later rows explore the related request and confirmation behavior. They need not all become one piece of work.

When the club agrees on a result, we can record it as an **acceptance criterion**: a condition used to decide whether the story has been satisfied. (3) For the visibility story, one proposed criterion is: "When the Saturday shift needs two more volunteers, the schedule shows two unfilled places." That gives us a concrete check. It still leaves the pending-request rule to be resolved before we rely on the count during signup.

Stories help us find useful examples, and examples help us discover missing decisions. We do not need a complete collection of cards before beginning. Nor does every requirement need to become a story: privacy rules and reliability constraints can apply across many user experiences. Record those obligations where the team can find them too.

Needs, requirements, and design choices

Suppose a volunteer says, "I need to know that I'm actually on the schedule."

That is a need. It expresses what matters to the person without choosing how the software will meet it.

We might propose this requirement:

> After a signup is confirmed, the volunteer can view the event, date, and shift time associated with that commitment.

The statement gives us something more definite to build and discuss. It still leaves room for design choices. The confirmation could appear on a page, in an email, or in both places, depending on what else we learn.

Now compare it with "Put a green checkmark beside the volunteer's name." That describes a proposed interface. It might contribute to a solution, but it leaves important questions unanswered. Who can see the checkmark? What does it mean? Can the volunteer find it later?

Design choices can also be real requirements when there is a reason for them. An organization may need a particular sign-in service because it manages membership there. A required export format may come from an existing system that consumes the file. The distinction depends on the reason behind the statement, not just the kind of words it contains.

For the club, "Use the organizer's personal account for everything" deserves investigation. It is convenient today, but it may make the application difficult to hand over. The underlying need might be simple administration. We should preserve that need while considering a better way to meet it.

What the software does and how well it works

A useful way to begin thinking about functional requirements is to ask what the product must do to be the kind of thing we say it is. A car that cannot ever move under its own power does not fulfill the basic purpose of a car. It is a box. Movement is part of what we are buying when we buy a car.

Now consider another requirement: the car must not catch fire when it hits a wall at parking-lot speed. A vehicle that fails this requirement might still be a car. It would, however, be a very bad one. Being able to perform its basic function is not enough to make it acceptable.

A **functional requirement** describes behavior the product must provide (5). In software, that includes the central activities that make the application useful, as well as supporting behavior. For the club, candidate requirements might let a volunteer request a shift, let an organizer close signups, and update available capacity when a commitment is canceled. These are proposals to discuss with the club. Not every individual function defines the whole product, but each describes behavior we could agree to provide.

Nonfunctional requirements often express constraints on an acceptable product: it must be a car, but it also must meet expectations for safety. For the signup application, we might require that confirmed commitments survive a restart. These requirements can concern qualities such as reliability or usability, and are often called **quality requirements** (6).

The same distinction appears in familiar products. Read each row as “It must perform this function, but it also must satisfy this constraint.” These are examples of requirements we might set for a product, rather than complete specifications.

Product	Basic function	Safety or environmental constraint
Car	Transport people	Must not catch fire in a collision at parking-lot speed.
Children’s pajamas	Provide clothing for sleep	Must not be flammable.
Candy bar	Provide a sweet food to eat	Must not contain mold.
Nerf gun	Fire foam darts	Must keep dart velocity below a specified safety limit.
Rocket	Carry a payload into flight	Must not drop toxic fuel on the ground.

Consider the pajamas. If they are flammable, they can still be worn to bed. They can perform their function and still be unacceptable. Is the flammability requirement functional or nonfunctional? Nonfunctional. Is it important? Very important.

Requirement type and requirement importance are separate questions. Calling a requirement nonfunctional does not make it optional.

These short statements give us a starting point for discussion. Before testing a product, we would need to clarify the conditions and how compliance will be checked. For the foam-dart toy, for example, we need an agreed velocity limit and a way to measure it. That detail makes the requirement testable; it does not make it more or less functional.

The categories are useful, but their boundaries are not always sharp. Controlling who may cancel a commitment is both a question about behavior and a security concern. We need to understand the obligation more than we need to win an argument about its label.

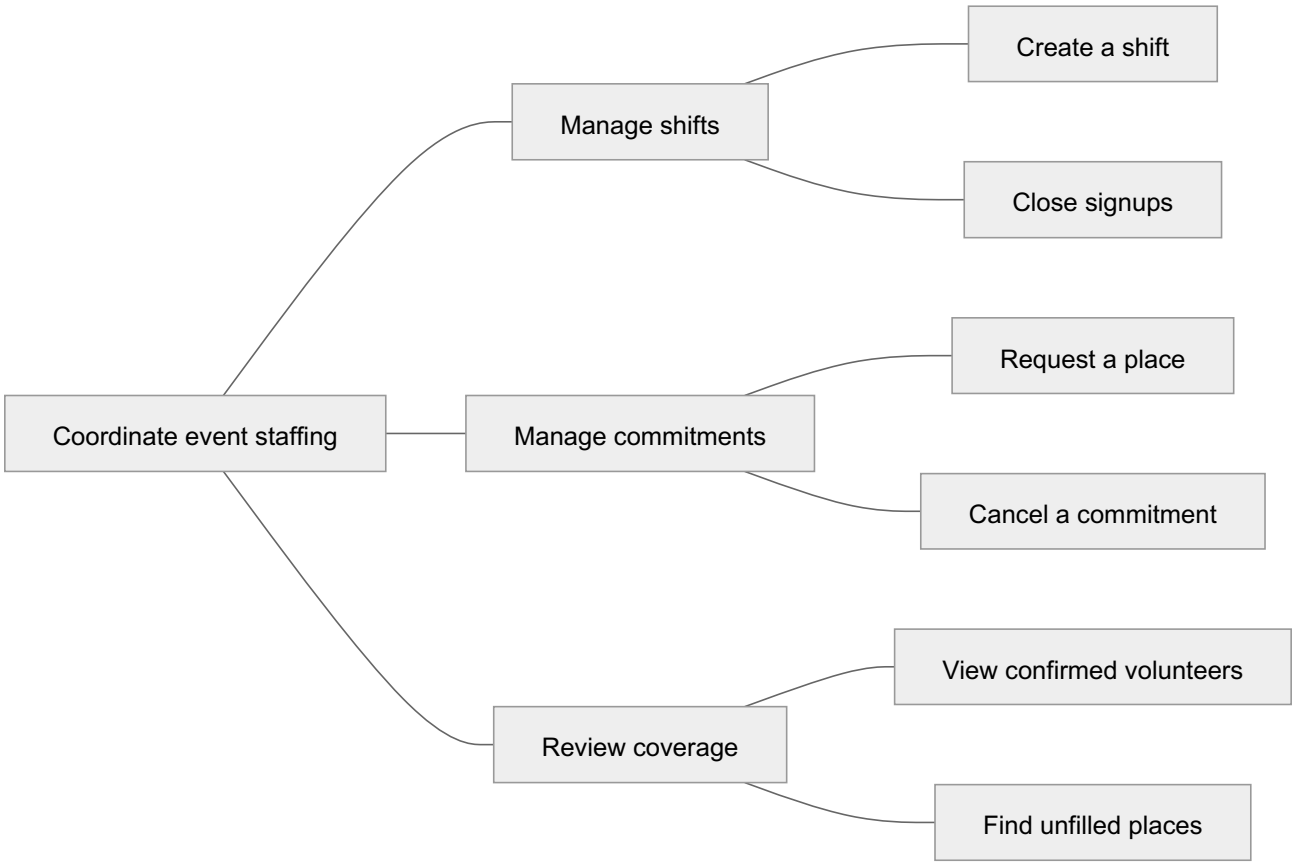
Vague descriptions are especially common when people discuss quality. “Fast,” “secure,” and “easy to use” can conceal very different expectations. Make the concern concrete enough to investigate.

Initial request	A question that makes it clearer
Make it easy to use.	Can a new member request a shift and later check whether it was confirmed, using their phone without help?
Make it reliable.	What must remain saved if the application restarts after confirmation?
Make it private.	Which details can volunteers see, and which are only for the organizer?
Make it fast.	Which action is too slow, under what conditions, and how much delay is acceptable?

These questions do not supply the answers. In particular, a developer should not invent a performance target just to make a statement look precise. The people who depend on the system need to help establish what matters, and the team needs to determine whether it can be achieved.

Seeing the functions together

A **function hierarchy** breaks a broad capability into smaller activities, an application of functional decomposition (7). Here is an initial view of what the signup application might need to support.



Read the chart from the broad purpose on the left toward the more specific functions on the right. Each connecting line means “includes.” It does not mean “do this next.” Finding an unfilled place is part of reviewing coverage; it is not a step that must follow viewing confirmed volunteers.

The hierarchy helps us notice gaps and discuss scope. An application could accept requests while leaving the organizer unable to find an uncovered shift. Seeing both branches makes that omission easier to catch. We can also discuss which functions are needed for the next event and which can wait.

This is a view of the work the application supports, not a plan for its screens or code modules. It is also allowed to be incomplete. Add detail when a question makes that detail useful.

Clearing up the language

Consider the statement “Members can cancel their shifts.”

It sounds simple. Does it mean any member can cancel any shift? Can someone cancel after the event begins? Does canceling remove their commitment, or does it remove the shift itself from the event?

A more careful version might be:

> A volunteer can cancel their own confirmed commitment before the event begins. The shift remains on the schedule, and its available capacity is updated.

This is a proposed rule for the example, not a rule we have already established with the club. Its value is that people can now react to something specific. The organizer might need a different cutoff or a way to handle late cancellations. We have a question to resolve.

Consistent terms help. If "request" means an offer to volunteer and "confirmation" means an accepted commitment, use those meanings throughout the discussion. Otherwise one person may believe a request reserves a place while another believes it is waiting for approval.

Look for statements that combine several unresolved decisions. "Automatically notify everyone whenever anything changes" leaves the recipients, the triggering changes, and the delivery method uncertain. It also risks making the application irritating enough that people ignore its messages.

Precision does not require a large document. A few clear sentences and an example can be enough to expose a disagreement. The amount of detail should help the people doing the work make consistent decisions.

When reasonable needs conflict

Volunteers may want the freedom to cancel close to an event. The organizer may need a dependable schedule a day in advance. Both needs are understandable. The application cannot make that tension disappear.

The club has to decide what arrangement it can support. Perhaps volunteers can cancel directly until a cutoff, then contact the organizer afterward. Perhaps cancellations remain possible at any time, but the organizer is alerted immediately. Each choice has consequences for the people using it.

A developer can explain those consequences and suggest alternatives. The club needs to make the operating decision. Quietly choosing whichever behavior is easiest to implement leaves the disagreement hidden until somebody encounters it.

Recording the decision matters. Include the reason when it is likely to come up again. "Late cancellations go to the organizer because replacements must be arranged directly" is more useful than a bare restriction that nobody remembers agreeing to.

Unresolved questions should stay visibly unresolved. If the club has not chosen whether signup requires approval, a polished screen should not give the impression that approval has been settled. An honest open question is easier to manage than an accidental promise.

Constraints and deadlines

A **constraint** limits the solutions available to us. For the club, the next event is in ten days and there is no budget for a paid service. Existing account access may also limit where an application can run.

Some apparent constraints are preferences. "We have always used this tool" may describe familiarity rather than a requirement. Ask what makes the choice necessary. There may be a good reason, such as an existing support arrangement, or there may be room to choose differently.

A deadline needs similar attention. What happens on that date? A live event creates a different situation from an internal target that can move. We also need to know what must be usable by then. "The application is due Friday" tells us less than "Volunteers need to make confirmed commitments before staffing closes Friday afternoon."

If the necessary work does not fit, the team needs a decision. It could reduce what it delivers, arrange more help, or use an existing arrangement for the event. Adding people late may introduce its own coordination work. A deadline does not remove that difficulty.

An early conversation about what can be delivered is part of responsible engineering. It gives the people depending on the work time to choose an alternative. Waiting until the deadline to reveal the problem takes choices away from them.

Why the work is worth doing

A **business need** explains why an organization should spend effort on the solution. The word business applies here even though the club is not selling a product.

The club needs dependable event coverage without repeatedly consuming the organizer's evening. The proposed application is worthwhile only if it helps enough to justify the effort of building and operating it.

Compare it with the alternatives. A shared schedule may already solve much of the problem. A custom application might save work across many events, but it also needs maintenance. The meaningful comparison includes the work people will do after launch.

We can ask the organizer how much time they currently spend reconciling signups and resolving mistakes. Those observations give us a starting point for judging improvement. We should not turn a guess about time saved into a promise simply because it makes the proposal sound attractive.

A short statement of purpose can keep the work focused:

> Help club members make clear volunteer commitments and help the organizer see staffing gaps before an event, without maintaining conflicting signup lists.

That statement gives us a reason to question a proposed feature. Public member profiles might be interesting, but their connection to this purpose is weak. A clear record of a canceled commitment is much easier to justify.

The purpose can change when the club learns something important. A change deserves an explicit conversation, because it may affect work people have already agreed to do.

Checking the shared understanding

Writing down requirements does not establish that they are right. We need to check our account with the people involved.

Validation asks whether the proposed requirements describe a useful solution to the actual problem (8). If volunteers cannot tell whether a request has been accepted, a perfectly implemented request form may leave their difficulty unresolved.

Verification checks whether a result conforms to what was specified (8). If the agreed rule allows volunteers to cancel only their own commitments, the implementation needs to enforce that rule.

Both matter. A result can match an agreed description while the description itself misses the need. It can also address a real need but fail to behave as agreed.

Walk through ordinary situations with the organizer and a volunteer. Someone chooses the last available place. Someone changes their mind. An organizer closes an event. Ask each person what they expect to see and what they would do next. Disagreement is useful information while changes are still inexpensive. It does not always have to be resolved before anything can begin. Ask whether the answer affects the next piece of work or can wait until there is something concrete to examine.

The record should be easy to find and keep current. A decision in one person's private message is not a shared understanding. Neither is an old document that everyone knows is out of date but nobody has corrected.

This has a direct effect on the experience of developing software. Clear expectations reduce the chance of finishing work only to hear, "That isn't what I meant." Visible decisions also reduce repeated arguments and dependence on one person's memory. Good requirements work gives people a fairer chance to do something useful and take satisfaction in finishing it.

Considerations

1. A team agrees that volunteers need one shared schedule, but disagrees about reminders and approval rules. What useful work could begin now? Which unanswered question would you resolve first, and why?

2. Think of a product you use. What is one functional requirement and one nonfunctional requirement it must satisfy? Could failing the nonfunctional requirement make the product unacceptable even if its main function still works?

3. The organizer says, "We need a dashboard." What underlying need might that request express? What would you ask or observe before agreeing to build it?

4. Compare the signup workflow with the function hierarchy. What question can each chart help answer that the other does not? Identify one missing detail worth discussing before the next piece of work begins.

5. A team delivers every agreed feature, but the organizer still spends each evening reconciling signups. What would you investigate? How could the team have discovered the problem sooner without waiting for a complete set of requirements?

Practice

Work through the club's cancellation question

The organizer says, "People should be able to cancel, but I can't have everyone disappearing just before the event."

Write a short response that makes the decision clearer:

1. Identify whose needs are involved and where they conflict.
2. Propose a cancellation rule in plain language. Mark it as a proposal.
3. Describe one situation that would reveal whether the organizer and a volunteer understand the rule the same way.
4. Identify one quality concern and one practical constraint that could affect the solution.
5. Explain how the proposed rule supports the club's reason for doing the work.

You do not need a screen design or a choice of technology yet. You need a description that another person can examine, question, and improve. Aim for enough shared understanding to take a useful next step. The description can be revised as people learn more.

Write a story card

Write a card for a volunteer who needs to check whether a request has been approved. Include the role, the useful capability, and the reason it matters. Add one concrete situation with an observable result. Record one open question instead of silently choosing its answer.

References

1. Wikipedia contributors. “Requirements elicitation.” https://en.wikipedia.org/wiki/Requirements_elicitation.
2. Wikipedia contributors. “Workflow.” https://en.wikipedia.org/wiki/Workflow.
3. Wikipedia contributors. “User story.” https://en.wikipedia.org/wiki/User_story.
4. Agile Alliance. “The Three C’s.” https://agilealliance.org/glossary/three-cs/.
5. Wikipedia contributors. “Functional requirement.” https://en.wikipedia.org/wiki/Functional_requirement.
6. Wikipedia contributors. “Non-functional requirement.” https://en.wikipedia.org/wiki/Non-functional_requirement.
7. Wikipedia contributors. “Functional decomposition,” Systems engineering section. https://en.wikipedia.org/wiki/Functional_decomposition#Systems_engineering.
8. Wikipedia contributors. “Verification and validation.” https://en.wikipedia.org/wiki/Verification_and_validation.